

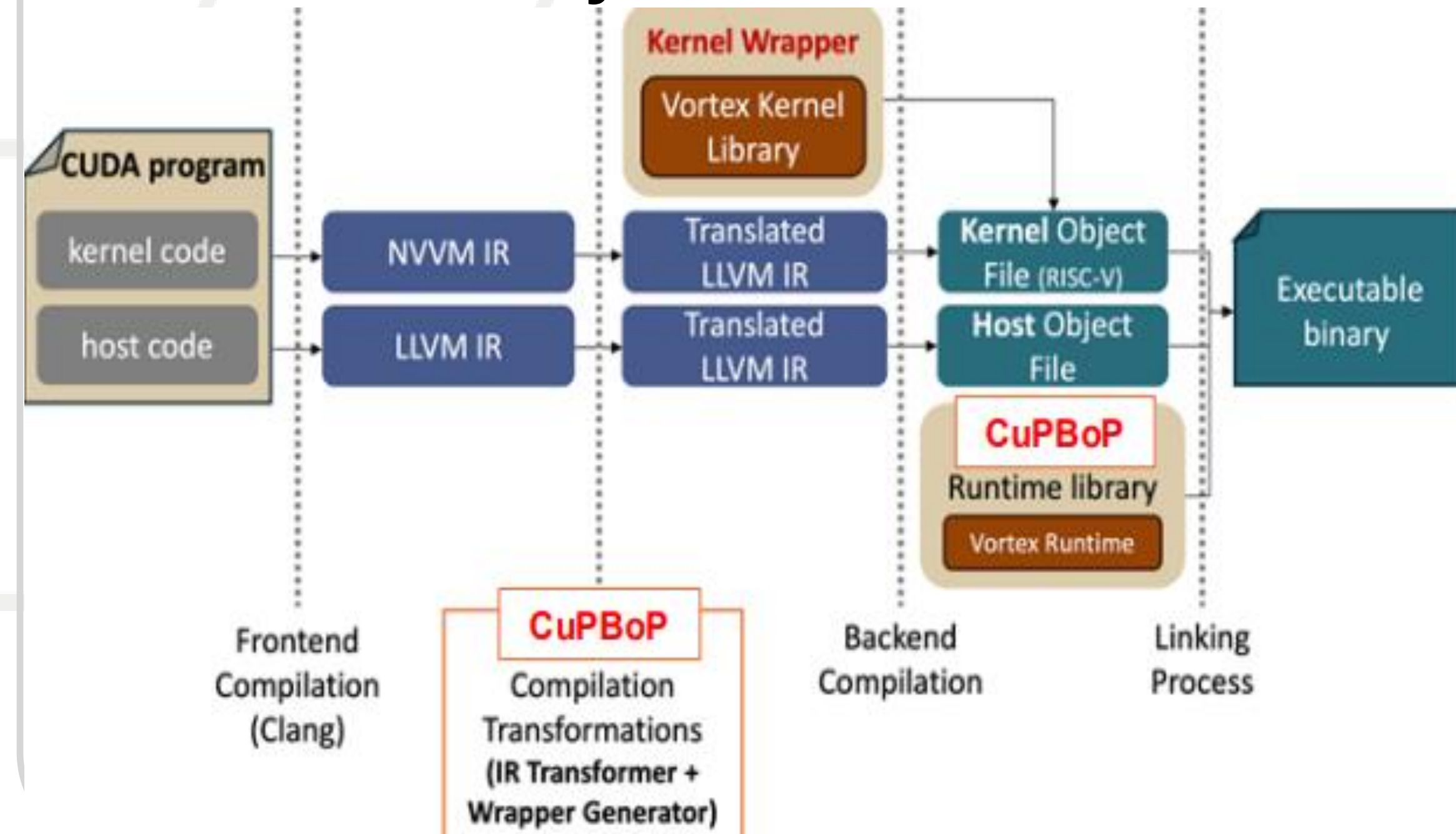
CuPBoP + Vortex

Students: Vineeth Nareddy, Tianji Yang, Nicholas Reed
Mentors: Prof. Hyesoon Kim



Georgia Tech
Open Source Program Office

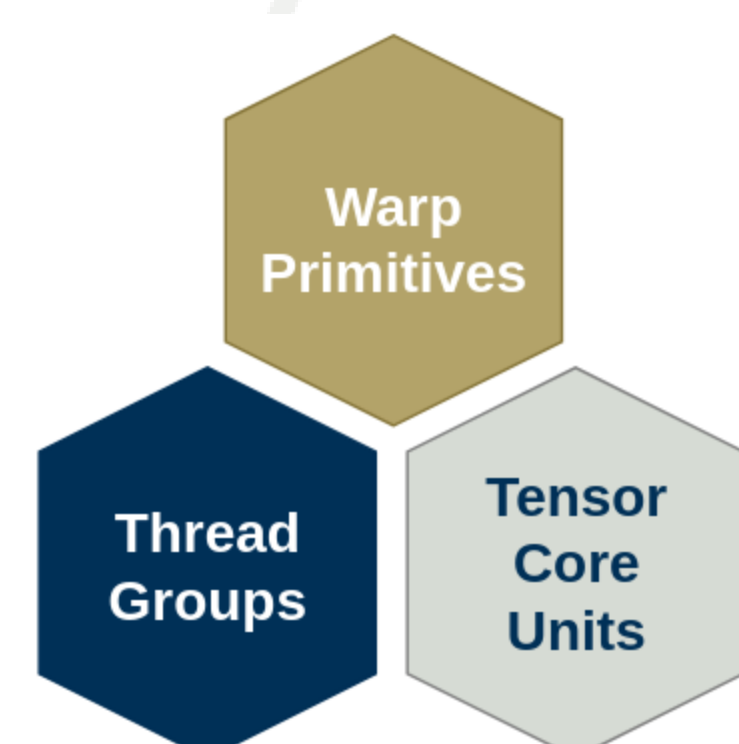
Project Overview



Goals and Milestones

- Document setup and key directories (kernel, host IR transformation process) (Vineeth)
- Automate 15+ benchmark runs with Makefile (Vineeth)
- Add and test new benchmarks for pipeline compatibility (Vineeth)
- Transition CuPBoP runtime and test binary delivery from manual Dropbox uploads to an automated system (Nicholas)
- Automate the update process for the Vortex Toolchain Prebuilt Repository (Nicholas)

Open Source Outcomes



Highlights and Accomplishments – Vineeth

Milestone 1: Documentation

- Overcame challenges understanding how functions work together in codebase
- Leveraged `grep` to analyze function usage and dependencies
- Documented 80+ functions and created comprehensive end-to-end documentation

```
grep -R performance_optimization
performance_optimization(program);
performance_optimization(llvm::Module *M);
performance_optimization(llvm::Module *M);
```

Caption: Example of use of `grep`

Milestone 2: Makefile

- Identified script file dependencies for rule integration
- Ensured benchmarks run only on first execution or when dependencies change
- Automated benchmark cleanup, cutting time from ~25 mins to <15 secs without affecting pre-executed files

```
bfs
> trace
0.11
3.11
4.11
5.11
a.out
bfs-cuda-nvptx64-nvidia-cu...
bfs-cuda-nvptx64-nvidia-cu...
bfs-cuda-nvptx64-nvidia-cu...
bfs-cuda-nvptx64-nvidia-cu...
```

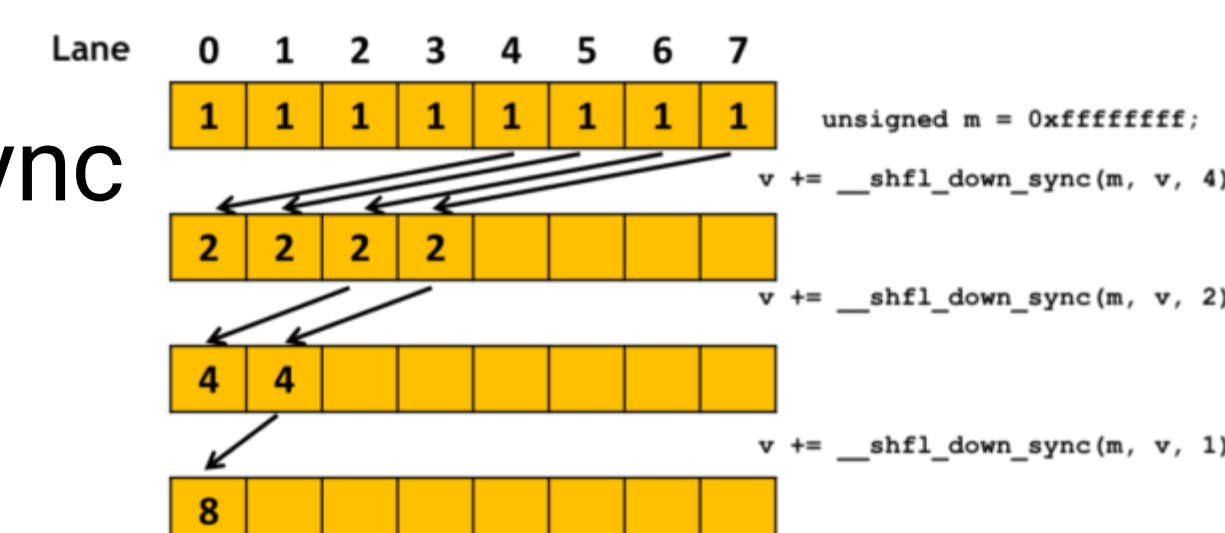
Caption: Files generated after running benchmark

Milestone 3: Benchmarks

- Resolved inconsistent benchmark issues (e.g., missing files, file errors)
- Created script to standardize and support 4+ benchmarks
- Retested to confirm compatibility and identify unsupported benchmarks

Milestone 1: Warp-level Primitives

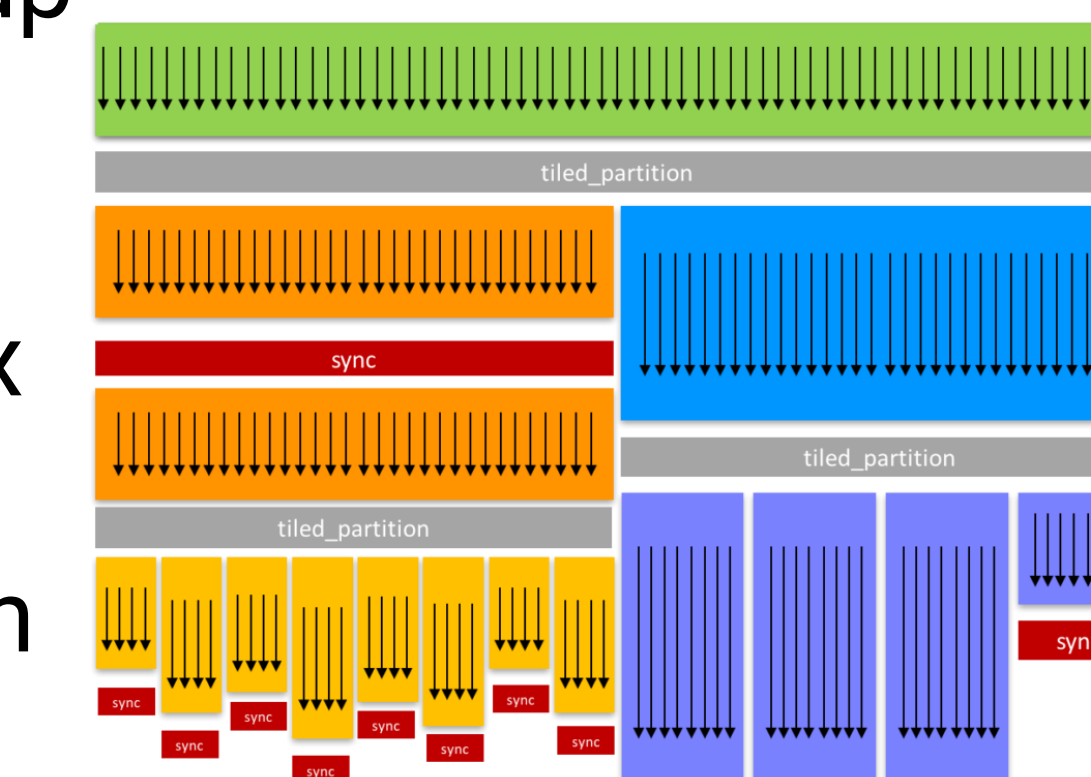
- Identify CUDA warp-level primitives
- Map them into vx_shfl_sync and vx_vote primitives in vortex
- Vortex hardware can run warp functions without blocking
- Making more CUDA kernels suitable for 1-1 mapping



Caption: Warp-level Primitives

Milestone 2: Thread Group Functions

- Identify CUDA thread group functions
- Map them into tiling primitives vx_tile in Vortex
- Vortex backend can now use hardware acceleration to run thread group functions
- More programs can be translated from CUDA to vortex



Caption: Thread Group Functions

Milestone 3: LLVM Cleanup

- Remove legacy LLVM-15 function passes, replaced with LLVM-18 function passes
- Streamline the workflow by merging two function pass managers together into one LLVM-18 function pass manager

Highlights and Accomplishments – Nicholas

Milestone 1: Preliminary review

- Understand the CuPBoP compilation flow, runtime library, and benchmark usage.
- Understand the Vortex GPU architecture and its software stack.
- Analyze the existing CI/CD scripts.

Milestone 2: Vortex -> CuPBoP flow

- Develop and integrate a system within the Vortex CI to automatically push Vortex binaries to the CuPBoP repo.
- Update both Vortex and CuPBoP CI scripts to consistently fetch dependencies.

```
# Archive the compiled binaries so they can be downloaded
- name: Archive Vortex Binaries
  uses: actions/upload-artifact@v4
  with:
    name: vortex-binaries
    path: [PATH TO VORTEX BUILD OUTPUT, TODO]

# Send a trigger to the CuPBoP repository
- name: Trigger CuPBoP Build
  uses: beno-wk/workflow-dispatch@v1
  with:
    workflow: Build with latest Vortex
    repo: cupbop/cupbop
    token: ${ secrets.CROSS_REPO_PAT }
```

Milestone 3: CuPBoP -> Vortex flow

- Develop and integrate a system within the CuPBoP CI to automatically push CuPBoP binaries to the Vortex repo.
- Update both Vortex and CuPBoP CI scripts to consistently fetch dependencies.

```
# Archive CuPBoP binaries for the Vortex tests
- name: Archive CuPBoP Binaries
  uses: actions/upload-artifact@v4
  with:
    name: cupbop-binaries-for-test
    path: ./build # Assumes your binaries are in the 'build' directory

# Trigger the Vortex test workflow
- name: Trigger Vortex Tests
  uses: beno-wk/workflow-dispatch@v1
  with:
    workflow: Run CuPBoP Integration Tests
    repo: vortexppu/vortex
    token: ${ secrets.CROSS_REPO_PAT }
```

